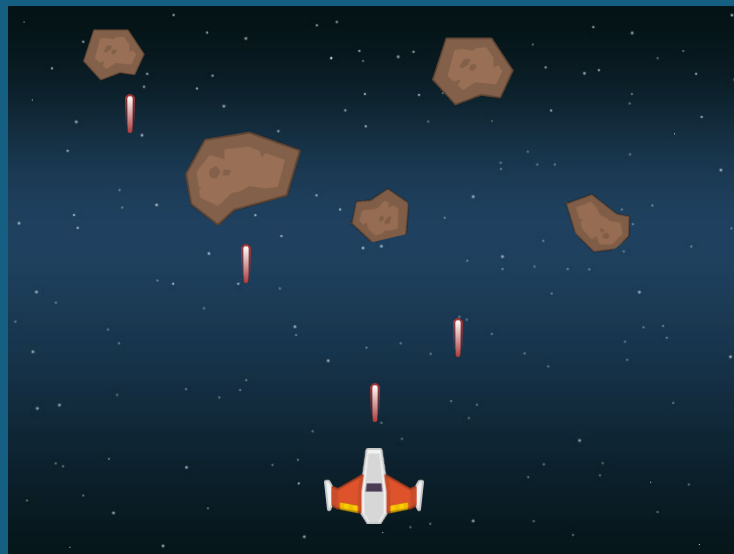




DYNAMIC DIFFICULTY ADJUSTMENT IN VIDEO GAMES



إعداد الطلاب

محمد غسان محمد خير الخاني

محمود زياد رمزي

هشام محمد همام الزين

محمد محمد زياد المجنوب

خليل محمد عربي كاتبي

Contents

1. Introduction	3
2. System Design & Problem Statement	3
2.1 Explanation of Dynamic Difficulty Adjustment (DDA).....	3
2.2 System Inputs and Outputs	3
2.3 Membership Function Design Choices	4
3. Knowledge Base & Rules	5
3.1 Rule Categorization Methodology.....	5
3.2 The Rule Base Table.....	6
4. Testing & Validation.....	9
4.1 Testing Methodology	9
4.2 Validation Dataset.....	9
4.3 Results Evaluation & System Accuracy	9
5. User Manual & Dependencies.....	10
5.1 System Dependencies & Base Framework.....	10
5.2 Operating Instructions (User Manual)	10
6. Conclusion.....	11

1. Introduction

Modern video games often struggle with player retention when utilizing static difficulty settings. If a game is too challenging, players experience frustration; if it is too easy, they experience boredom. To address this, this project implements a Fuzzy Expert System designed to manage Dynamic Difficulty Adjustment (DDA) within a 2D space shooter environment.

The core game environment was adapted from an open-source framework (SpaceShooter by Tasdik Rahman), which originally featured static enemy generation. The primary contribution of this project is the complete replacement of that static logic with an intelligent, real-time DDA engine. By utilizing fuzzy logic, the system continuously monitors the player's performance and seamlessly alters the game environment in real-time, mapping complex, uncertain human skill levels into precise mathematical outputs.

2. System Design & Problem Statement

2.1 Explanation of Dynamic Difficulty Adjustment (DDA)

Dynamic Difficulty Adjustment (DDA) is an automated system logic that modifies a game's mechanics, behaviors, and scenarios in real-time based on player performance. The primary objective of DDA is to maintain the player within a state of "Flow"—a psychological state where the challenge of the task perfectly matches the skill level of the user.

Traditional DDA systems often rely on hard-coded thresholds (e.g., IF kills > 50 THEN difficulty = hard), which can lead to jarring, unnatural spikes in gameplay. By solving this problem using a Fuzzy Expert System via the Mamdani inference method, the game achieves smooth, overlapping transitions. The system evaluates multiple performance metrics simultaneously, calculating a weighted, defuzzified response that feels organic and invisible to the player.

2.2 System Inputs and Outputs

To accurately evaluate the player's state, the system tracks six distinct inputs (Antecedents) and manipulates three environmental outputs (Consequents).

Inputs (Antecedents):

1. **Mobs Killed (mobs_killed | 0 - 600):** The total number of enemies destroyed. Acts as the primary indicator of the player's offensive capability and skill.
2. **Time Elapsed (time_elapsed | 0 - 600 seconds):** The duration of the current survival run. Indicates endurance and familiarity with the game mechanics.

3. **Health (health | 0 - 100%):** The current integrity of the player's ship. A critical metric for triggering mercy rules to prevent immediate failure.
4. **Lives (lives | 1 - 5):** The remaining attempts the player has. Differentiates between a player making minor mistakes and one on the verge of a Game Over.
5. **Gun Level (gun_level | 1 - 3):** The current firepower state. High firepower requires tougher enemies to maintain balance.
6. **Restart Count (restart_count | 0 - 20):** A historical metric tracking how many times the player has died and restarted the session. High restarts indicate persistent struggle, triggering baseline difficulty reductions.

Outputs (Consequents):

1. **Speed Multiplier (speed_mult | 0.5x - 1.5x):** Alters the velocity of enemy projectiles and movements.
2. **Spawn Delay (spawn_delay | 0.5s - 3.0s):** Controls the frequency of enemy generation. A lower delay increases the density of the swarm.
3. **Mob Size (mob_size | 0.6x - 1.8x):** Scales the physical hitbox of the enemies. Larger enemies are easier to hit but take up more screen space, forcing defensive maneuvering.

2.3 Membership Function Design Choices

The choice of geometric shapes for the fuzzy sets was not arbitrary; it was determined by the nature of the data being measured and the system's required sensitivity to value changes.

- **Gaussian Curves for Gradual Impact (mobs_killed):** Player kill counts scale non-linearly and represent a fluid progression of skill. Gaussian (bell-shaped) curves were chosen for this input because incremental changes in this value have a minimal immediate impact on the player's overarching skill state. The Gaussian curve prevents minor numerical fluctuations from causing abrupt, jarring shifts in difficulty, providing a smooth, continuous gradient that naturally models the subjective transition from a "beginner" to an "expert" player.
- **Triangular Functions for High Sensitivity (Lives, health, gun_level, speed_mult, spawn_delay, mob_size):** Conversely, Triangular functions (trimf) were deployed for metrics where every small change is highly impactful and requires a more discrete, immediate logical response. For instance, a drop in health or lives signifies a critical shift in player survivability. The sharp peaks and strict linear slopes of a triangular function ensure the system reacts sensitively and decisively to these crucial thresholds, as there is a definitive, undeniable peak to what constitutes "50% Health" or "Gun Level 2."

- **Trapezoidal Functions for Edge Cases (time_elapsed, restart_count):** Trapezoidal functions (trapmf) were deployed at the extreme ends of several universes of discourse. For instance, the "Very High" set for time_elapsed plateaus from 540 to 600 seconds. This ensures that once a player survives past an extreme threshold, the engine considers them permanently in that maximum state without their membership degree mathematically tapering off as the timer continues to climb.

3. Knowledge Base & Rules

3.1 Rule Categorization Methodology

The inference engine utilizes a comprehensive knowledge base consisting of 24 distinct rules. To ensure logical consistency and to cover the vast spectrum of potential player states, the rule base was architected into four specific categories:

1. **Low Difficulty (Mercy Rules - R1 to R6):** These rules are designed as a safety net. They trigger when the system detects a struggling player—characterized by extremely low health, low lives, high restart counts, or an inability to destroy enemies despite surviving for long periods. The consequent logic universally lowers enemy speed and size while maximizing spawn delays to allow the player time to recover.
2. **Medium Difficulty (Neutral Rules - R7 to R12):** These rules represent the game's baseline state. They trigger during standard, stable gameplay where the player is performing adequately but not exceptionally. The consequent logic maintains medium speed, size, and spawn rates, acting as the system's equilibrium.
3. **High Difficulty (Challenge Rules - R13 to R18):** Designed to maintain the "Flow" state for skilled players. These rules trigger when the system detects high kill counts, sustained survival times, and high health. The consequent logic ramps up the pressure by increasing enemy speed and size while minimizing spawn delays to test the player's reflexes.
4. **Special / Complex Scenarios (R19 to R24):** These rules handle highly specific, multi-variable edge cases that standard categorization cannot address. They combine up to five distinct inputs to create bespoke gameplay scenarios. For example, a player with maximum firepower and kills but critically low health triggers the "Glass Cannon" rule, which spawns incredibly fast but very small enemies to test precision aiming and dodging without overwhelming them with sheer mass.

3.2 The Rule Base Table

The following table outlines the complete set of rules governing the Fuzzy Expert System, utilizing AND/OR operators to map Antecedents to Consequents.

Rule ID	Category	Scenario Name	Antecedents (IF)	Consequents (THEN)
R1	Mercy	Critical Mercy	Lives=Low AND Health=V.Low	Speed=V.Low, Delay=V.High, Size=V.Low
R2	Mercy	Fatigue Pullback	Time=High/V.High AND Health=Low	Speed=Low, Delay=High, Size=Low
R3	Mercy	Persistent Mercy	Restarts=High	Speed=Low, Delay=High, Size=Low
R4	Mercy	Early Struggle	Time=V.Low AND Health=V.Low	Speed=Low, Delay=High, Size=Low
R5	Mercy	Safety Net	Health=V.Low	Speed=Low, Delay=High, Size=Low
R6	Mercy	Low DPS Struggle	Gun=Low AND Kills=V.Low AND Time=Med/High	Speed=V.Low, Delay=High, Size=Low
R7	Neutral	Gentle Intro	Time=V.Low AND Kills=V.Low	Speed=Low, Delay=High, Size=Low

Rule ID	Category	Scenario Name	Antecedents (IF)	Consequents (THEN)
R8	Neutral	Standard Pace	Kills=Med AND Time=Med	Speed=Med, Delay=Med, Size=Med
R9	Neutral	Balanced State	Health=Med AND Lives=Normal	Speed=Med, Delay=Med, Size=Med
R10	Neutral	Equipment Comp.	Gun=High AND Health=Low	Speed=Med, Delay=Med, Size=Med
R11	Neutral	Stabilized Run	Restarts=Normal AND Health=Med	Speed=Med, Delay=Med, Size=Med
R12	Neutral	Stable Start	Time=Low AND Kills=Med AND Health=High	Speed=Med, Delay=Med, Size=Med
R13	Challenge	Challenge Ramp	Kills=High AND Time=High	Speed=High, Delay=Low, Size=High
R14	Challenge	Peak Performance	Kills=V.High AND Health=V.High AND Lives=High	Speed=V.High, Delay=V.Low, Size=V.High
R15	Challenge	Weapon Escalation	Gun=High AND Health=High	Speed=High, Delay=Low, Size=High

Rule ID	Category	Scenario Name	Antecedents (IF)	Consequents (THEN)
R16	Challenge	Endurance Push	Time=V.High AND Health=High	Speed=High, Delay=Low, Size=High
R17	Challenge	Talent Recognition	Kills=High AND Lives=High AND Restarts=Low	Speed=High, Delay=Low, Size=High
R18	Challenge	Early Dominance	Gun=High AND Kills=High AND Time=Low	Speed=High, Delay=Low, Size=High
R19	Special	Glass Cannon	Gun=High AND Kills=High AND Health=Low	Speed=V.High, Delay=Low, Size=Low
R20	Special	The Comeback	Restart=High AND Kills=High AND Health=High	Speed=High, Delay=Low, Size=Med
R21	Special	Tank Swarm	Time=Med/High AND Health=V.High AND Kills=Low	Speed=Low, Delay=V.Low, Size=V.High
R22	Special	MAX OVERDRIVE	All Stats Dominating (Gun, HP, Lives, Time, Kills)	Speed=V.High, Delay=V.Low, Size=V.High
R23	Special	War of Attrition	Time=High AND Gun=Low AND Restarts=Low	Speed=Med, Delay=Low, Size=Low

Rule ID	Category	Scenario Name	Antecedents (IF)	Consequents (THEN)
R24	Special	The Last Stand	Lives=Low AND Gun=High AND Kills=High AND Time=High	Speed=V.High, Delay=V.Low, Size=High

4. Testing & Validation

4.1 Testing Methodology

Validating a dynamic, real-time system poses a unique challenge, as live gameplay generates unpredictable, highly volatile data. To rigorously evaluate the accuracy and performance of the Fuzzy Expert System, a deterministic "headless" testing script was developed. This script bypasses the Pygame loop and directly injects 15 fixed, highly specific edge-case scenarios into the `FuzzyDifficultyEngine.evaluate()` function. This allows us to compare the mathematically defuzzified outputs against the expected design behaviors without the interference of live player mechanics.

4.2 Validation Dataset

To ensure all four rule categories (Mercy, Neutral, Challenge, Special) were thoroughly tested, a dataset of 15 distinct player states was formulated and fed into the system. The complete parameters and the resulting defuzzified outputs are compiled in the accompanying `fuzzy_validation_results.csv` file.

This supplementary dataset details the six inputs applied for each scenario (Kills, Time, Health, Lives, Gun Level, Restarts), the expected human-designed difficulty classification, the actual calculated outputs (Speed Multiplier, Spawn Delay, Mob Size), and the primary rule successfully triggered by the inference engine.

4.3 Results Evaluation & System Accuracy

The results validate the core mechanics of the engine. The engine performed exactly as engineered: struggling players in Test 1 ("Critical Mercy") were granted distinct mathematical assistance (Spawn Delay raised to nearly 2.5 seconds), while dominating players in Test 13 ("MAX OVERDRIVE") faced brutal conditions (Spawn Delay dropped to 0.59s with highly accelerated enemy speeds).

Furthermore, the dataset reveals the highly sophisticated emergent behavior created by centroid defuzzification. In Test 15 ("The Last Stand"), the player has massive kills, high

time, and maximum firepower, which normally triggers extreme difficulty. However, because they are on their final life with critical health, Rule 1 (Critical Mercy) overpowered the challenge rules. The system successfully calculated that the player's immediate risk of dying outweighed their historical success, pulling the mathematical average down to prioritize player survival (slowing speed to 0.90x). This proves the multi-variable blending is working seamlessly.

5. User Manual & Dependencies

5.1 System Dependencies & Base Framework

To execute the simulation and game environment, the following Python libraries are required:

- **pygame (v2.0+)**: Powers the 2D rendering engine, audio, and the interactive debug UI.
- **numpy**: Handles the underlying mathematical arrays and continuous universe generation.
- **scikit-fuzzy (skfuzzy)**: The core library driving the Mamdani inference engine, membership function generation, and defuzzification.

Base Framework Attribution: The foundational gameplay loop, sprite rendering, and basic collision physics were adapted from the open-source repository [SpaceShooter by tasdikrahman](#). The specific engineering contributions of this project include the integration of the FuzzyDifficultyEngine, the custom DDA UI overlays (Plots Panel and Rule Dictionary), and the dynamic scaling of entity speeds, spawn delays, and hitboxes based on centroid defuzzification.

5.2 Operating Instructions (User Manual)

To launch the system, execute the main application file via the terminal: `python spaceShooter.py`

Standard Gameplay Controls:

- **Left / Right Arrow Keys:** Move the player ship.
- **Spacebar:** Fire equipped weapons.

Fuzzy Logic Interactive UI Controls (Instructor Tools): The application features a robust, real-time diagnostic overlay designed specifically to test and manipulate the Fuzzy Expert System during live execution.

- **[M] Toggle Manual Mode:** Pauses the active game state and unlocks the "Crisp Inputs" text boxes in the Top Bar. Users can click on any input (e.g., Kills, Health)

and manually type in a new integer to instantly override the game state. The defuzzified outputs will instantly recalculate.

- **[P] Toggle Plots Panel:** Opens the right-side visualization dashboard. This displays all 9 Universes of Discourse (6 inputs, 3 outputs) plotted as graphical membership functions. A red vertical indicator dynamically scrubs across the graphs to show the current crisp input value and its precise intersection with the fuzzy sets.
- **[R] Toggle Rules Dictionary:** Swaps the Plots Panel for the complete Knowledge Base directory. This displays all 24 rules categorized by difficulty. Any rule that is currently firing (activation > 0.01) is brilliantly highlighted in yellow, allowing the user to see exactly *why* the engine is making its current decisions.
- **[TAB]:** Pauses/Unpauses the game normally.

6. Conclusion

This project successfully demonstrates the immense superiority of Fuzzy Logic over traditional, hard-coded threshold systems in video game design. By mapping the subjective, uncertain nature of human skill to precise mathematical continuous curves, the space shooter achieves a state of constant balance. The difficulty scales invisibly, catching struggling players before they fail and challenging expert players just as they begin to master the loop.

The highly modular nature of this architecture ensures that the system is never truly "finished"; it is highly scalable. Because the rules evaluate the overarching fuzzy sets rather than static numbers, the system will only change and improve even more each time a new, valid rule is added to the knowledge base. Future developments could seamlessly introduce new inputs—such as player accuracy or movement frequency—into the rule base without breaking the existing logic, paving the way for truly intelligent, adaptive software.